

Spring Boot Annotations

1. @SpringBootApplication

- Firstly it use to mark java class as starting point of application
- And also this annotation is equivalent to three annotation which is **@EnableAutoConfiguration + @ComponentScan + @Configuration**
- Now in **@EnableAutoConfiguration** will Enable the Auto configuration means it will Automatically Configure if any Spring configuration based on the Jar dependency If any present in classpath so it will auto configured in our project.(We can use attribute “exclude” for exclude the specific auto configuration)
- For Example if we found spring-boot-starter-web Dependency in pom.xml then Spring boot application will automatically configure the web application context and JPA persistence context.
- **@ComponentScan** this is just a simple annotation where it will scan our All the Components is present in Specified package for example it will scan like @Component, @Service, @Repository and @Controller and then it will register as a Bean in Spring Application Context.
- If we need to scan out of the package so in that case we need to add @ComponentScan(basePackage = {"otherPackageComponent"}) in our Configuration class.
- **@Configuration** will help us to identifying Beans or class which will returning Beans for Spring Dependency Injection which will manage by the IOC Container. **In simple word this Annotation marks a class as source of Bean Definition.**

2. @Bean

- It will tell us this will returning the Bean to be managed by Spring Container.
- It will create a managed instance of particular return type of method.
- It is method level annotation.

3. @Component

- This annotation will tell us to this class is the Component which will manage by spring and we will use it with autowiring.
- It is class level annotation.

4. @Autowired

- This annotation is provide a feature which we can achieve Dependency Injection Automatically.
- It will inject automatically the Object from Spring Container by matching type of this particular Object.
- It help us to autowire the bean without creating an object using new keyword.
- We can apply to Constructor, Field, or Setter Method.
- @Autowired – DI
- @Qualifier – Ambiguity
- @Resource – DI + Ambiguity

Stereotype Annotation @Component, @Service, @RestController, @Controller, @Repository

5. @Controller

- This Annotation will use for define the class as component which will used a Restful web service to handle Requests and Responses.
- **@Controller** – Spring framework will always looking for HTML or view page for the returning string from method.
- If we annotated @Controller it will always expect as return type Model and View
- **@RestController** – It will return any thing we have in our method. This annotation is a combination of @Controller and @ResponseBody.

6. @Service

- This Annotation will use for define the class as component which will used for Service Layer.
- In Simple meaning this annotation is use for having the Business Logic of the application it this Specific Class.

7. @Repository

- This Annotation will use for define the class as component which will used for Represent the DAO layer of the application to deals with the database CRUD operations.

8. @Qualifier

- This annotation will Qualify the Specific Bean if there is more than One Bean is available.
- For example @Autowired annotation will check the type of Object and if there is two or more Same type of object is present then we will use @Qualifier annotation with Autowiring to avoid ambiguity problem.

9. @Lazy

- In spring boot a while defining spring bean then it will create Spring Beans in Container and now if we want this Spring bean is created as per requirement while we inject this bean then it will have to create then we will use @Lazy annotation.

10. @Value

- It will use for to inject the value explicitly from application.properties file with define key as attribute
- For example @Value("\${server.port}") – it will inject the port no. of application in define variable.

11. @PropertySource

- This annotation is use in Class level for defining the source of the properties which we will use by @Value attribute.

12. @ConfigurationProperties

- It will use for configure the specific properties form properties file.
- For Example:
In application.properties file

```
devname.name = abcd  
devname.age = 22
```

In somClass.class file

```
@ConfigurationProperties(prefix = "devname")  
public class somClass{  
    private String name;  
    private String age;  
}
```

- In this Example devname is the property in properties file and we will use or inject directly in Variable.

13. @Profile

- This will use for defining the which environment profile use for application.
- spring.active.profile = dev

14. @Scope

- If we define the scope of the bean we will use this annotation.
- For example we use this @Scope("singleton") or @Scope("prototype")

Scope of Bean :-

@Scope – It will tell scope of the bean means if we want to declare the scope of the Bean.
In that commonly we use singleton or prototype.

Type of Scope

Singleton – by default scope of spring bean – it will return same object reference every time in entire application.

Prototype – it will return always new object reference – it will create new object every time

Request – for HTTP request objects – it will create new instance for each HTTP request.

Session – for only session objects – it will create single instance of the bean per web socket session

15. @RestController

This is specialized version of Controller that include @ResponseBody means it directly return data rather than view.

It will return data in XML and Json format.

16. @RequestMapping

It will map the HTTP request to controller and define the URL path to controller method.

17. @GetMapping

18. @PostMapping

19. @PutMapping

20. @DeleteMapping

21. @RequestBody

It will bind the data from request into Java Object. It will convert JSON/XML from request body into java object.

22. @PathVariable

It will get the variable from URL path.

23. @RequestParam

It will get the parameter from request URL.

24. @ControllerAdvice

Provide global exception handling.

When we want handle Exception globally.

25. @ExceptionHandler

It will handle the specific and customize response for specific Exception and it will throw by controller method.

26. @Entity

- It will tell to JPA or Hibernate this class will perform to Database Related Operations.
- Use to mark any class as Entity.

27. @Table

- It will define to JAP or Hibernate to create the Table as the name provide and if we did not use this annotation the @Entity annotation will do automatically as per class name.

28. @Column

It specify the column name from class field and modify the column name.

29. @GeneratedValue

It will auto generate value for this column.

30. @GeneratorType

31. @Transactional

32. @Temporal

This will format the date.

33. @Transient

It will tell to hibernate not to save this class field.

34. @Data

It is Lambda annotation for generate getter, setter, toString, hashCode method. It use to avoid repetitive code.

35. @CrossOrigin

It will enable the CORS for application.

When we want different domain will allow to communicate with application.

36. @Secured

It will secure or restrict the specific method based on their role.

37. @PreAuthorize

38. @PermitAll

39. Caching Annotation

40. @EnableCaching

41. @Cacheable

42. @CachePut

43. @CacheEvict

AOP(Aspect Oriented Annotation)

44. @Aspect

45. @Pointcut

46. @AfterRunning

47. @AfterThrowing

48. @Around

49. @Before